

Contents

I. Overview.....	2
1. User Requirements	2
1.1 Actors	2
2. Overall Functionalities	2
2.1 Screens Flow.....	2
2.2 Screen Descriptions	8
2.3 Screen Authorization	8
2.4 State transition diagram of Request:	9
3. System High Level Design	11
3.1 Database Schema:	11
3.2 Description:	11
II. Requirement Specifications.....	13
1. Use Case Description	13
1.1 Use Case: Staff Send Request.....	13
1.2 Use Case: View Request List.....	17
1.3 Use Case: Manager Manage Request	18
1.4 Use Case: Manager View Request History	21
2. Common Functions	23
2.1 Use Case: Login System	23
III. Design Specifications	24
1. Screen design:	24
1.1 Screen: Login	24
1.2 Screen: Dashboard.....	26
1.3 Screen: Manager side: Manage Request:	28
1.4 Screen: Manager side: View Request History:	30
1.5 Screen: Staff side: Send Request:	31
1.6 Screen: Staff side: Request List:.....	33

I. Overview

1. User Requirements

1.1 Actors

Table 1: Actor

#	Actor	Description
1	Manager	Take responsible to manage request, change status
2	Staff	Create and send request to manager

2. Overall Functionalities

2.1 Screens Flow

a) Main screen flow

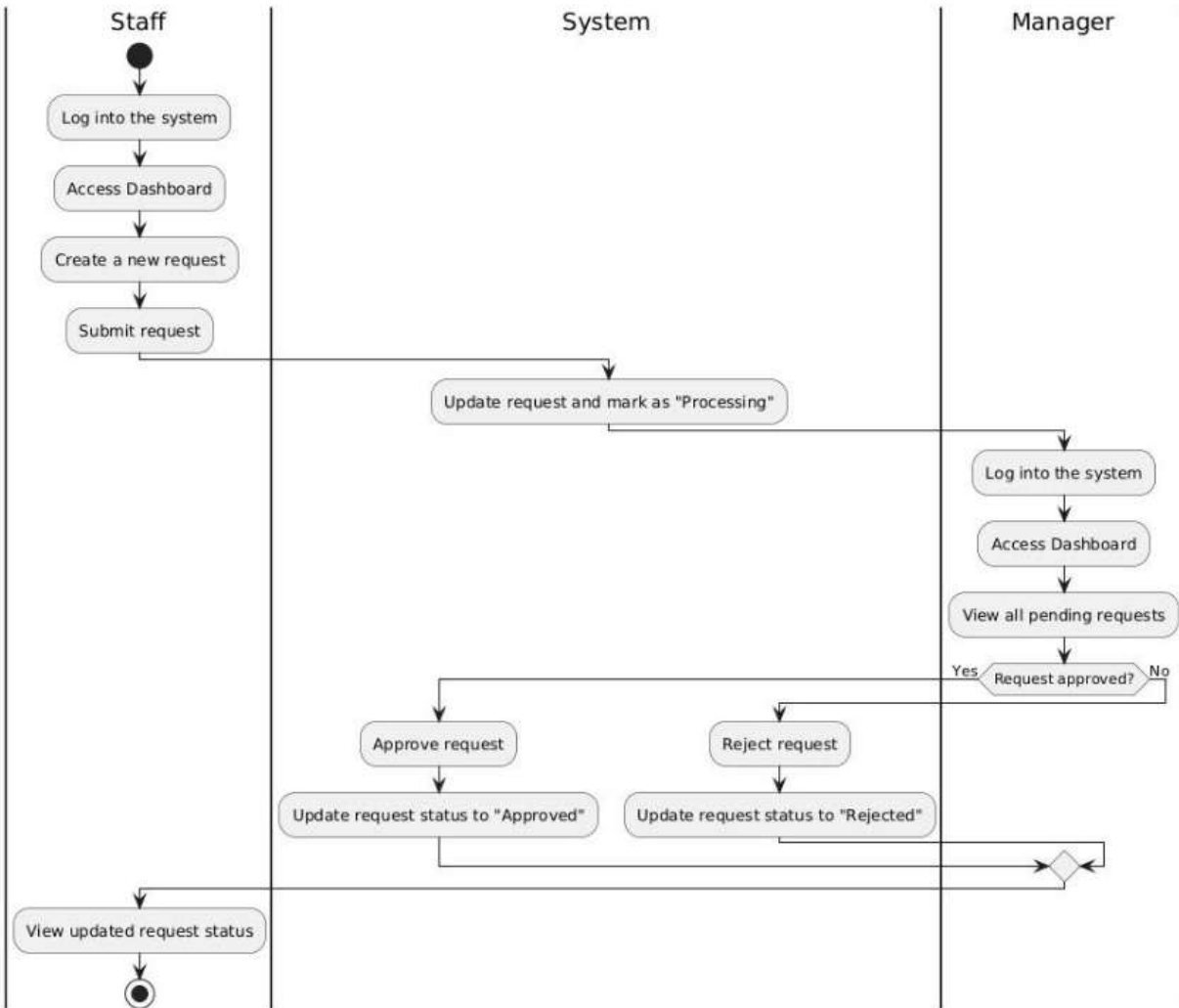


Figure 1: Activity diagram

- **Purpose:** To illustrate the overall flow of screens and actions in the system for both Staff and Manager roles.
- **Elements:**
 - **Swimlanes:** Staff, System, Manager
 - **Actions:** Log into the system, Access Dashboard, Create a new request, Submit request, Update request and mark as "Processing", Approve request, Reject request, Update request status to "Approved", Update request status to "Rejected", View updated request status, View all pending requests.
 - **Decision Point:** Request approved? (Yes/No)
- **Flow:**
 1. Staff logs into the system and accesses the dashboard.

2. Staff creates a new request and submits it.
3. The system updates the request status to "Processing". And Request sent to Manage Request
4. Manager logs into the system, accesses the dashboard, and views all pending requests.
5. Manager decides whether to approve or reject the request.
6. The system updates the request status accordingly ("Approved" or "Rejected").
7. Staff views the updated request status.

Description of main screen flow in json:

```
{ "diagram_name": "Main Screen Flow Diagram", "objects": [ { "object_name": "Staff", "type": "actor", "actions": ["Log into the system", "Access Dashboard", "Create a new request", "Submit request", "View updated request status"] }, { "object_name": "System", "type": "system", "actions": ["Update request and mark as 'Processing'", "Approve request", "Reject request", "Update request status to 'Approved'", "Update request status to 'Rejected'"] }, { "object_name": "Manager", "type": "actor", "actions": ["Log into the system", "Access Dashboard", "View all pending requests"] } ], "connections": [ { "source": "Staff", "target": "System", "action": "Submit request" }, { "source": "System", "target": "Manager", "action": "View all pending requests" }, { "source": "Manager", "target": "System", "action": "Approve/Reject request" }, { "source": "System", "target": "Staff", "action": "View updated request status" } ] }
```

Table 2: Activity diagram description

#	Feature	Screen	Actor	Description
1	Login	Login Screen	Staff, Manager	Staff and managers log into the system using their credentials.
2	Dashboard Access	Dashboard Screen	Staff, Manager	After login, users access the dashboard with options based on their roles.
3	Create Request	Request Submission	Staff	Staff members create a new request by entering details and submitting it.
4	Request Processing	System Backend	System	The system marks the request status as "Processing" automatically.
5	View Pending Requests	Request Management (Manager Side)	Manager	Managers can view a list of all pending requests submitted by staff.
6	Approve/Reject Request	Request Decision Screen	Manager	Managers review and take action on requests by approving or rejecting them.
7	Status Update	System Processing	System	The system updates the request status to "Approved" or "Rejected" based on the manager's decision.
8	View Request Status	Request List (Staff Side)	Staff	Staff can track their submitted requests and view updated statuses.

b) Staff screen flow

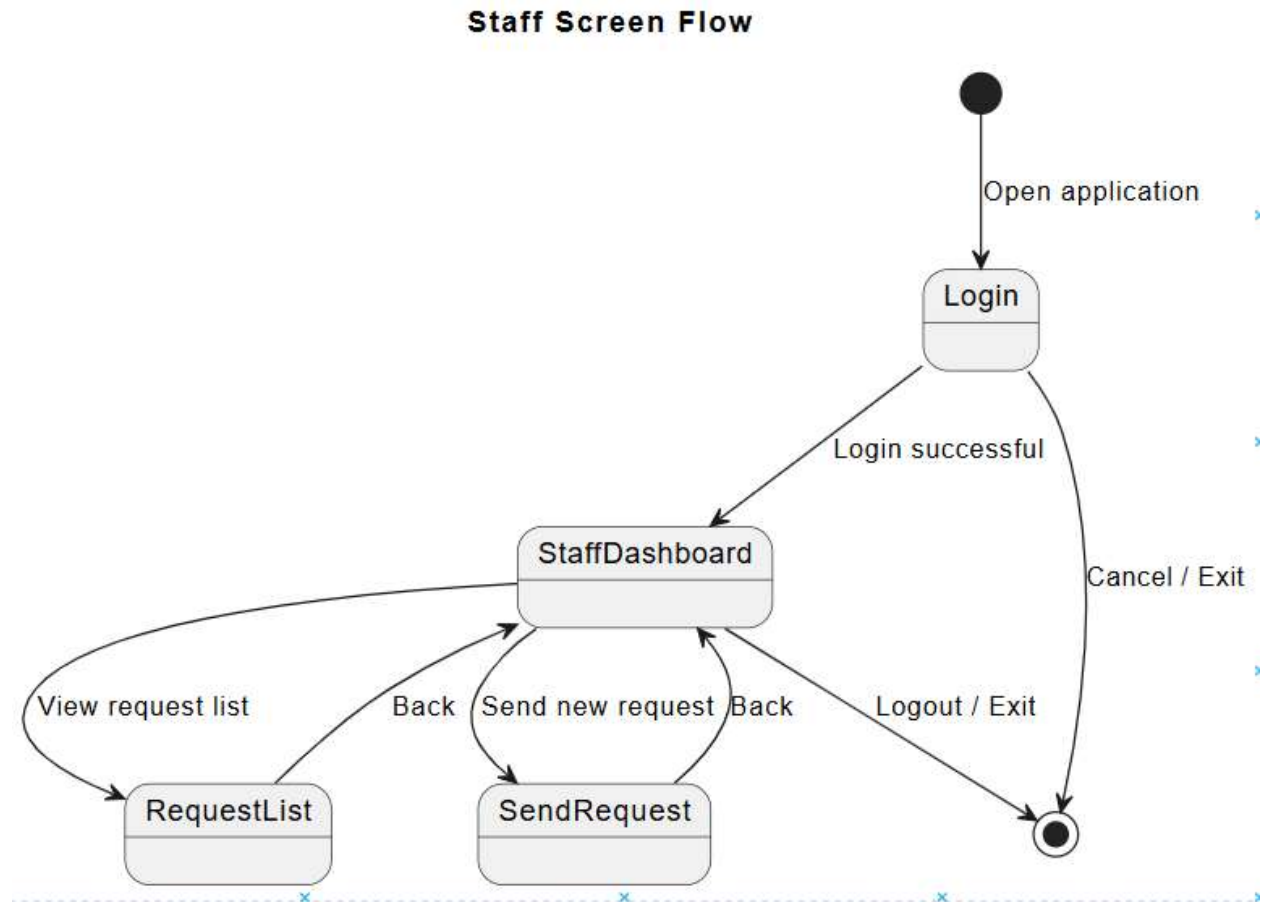


Figure 2: Staff Screen Flow diagram

- **Purpose:** To illustrate the screen flow specifically for the Staff role.
- **Elements:**
 - **Start Point:** Black circle
 - **End Point:** Black circle with a white dot in the center
 - **Screens:** Login, StaffDashboard, RequestList, SendRequest
 - **Actions:** Open application, Login successful, View request list, Back, Send new request, Logout / Exit, Cancel / Exit
- **Flow:**
 1. User opens the application.
 2. User logs in.
 3. Upon successful login, user is taken to the StaffDashboard.

4. From the StaffDashboard, the user can choose to "View request list" or "Send new request".
5. The user can navigate back to the StaffDashboard from either RequestList or SendRequest.
6. The user can "Logout / Exit" or "Cancel / Exit" the application.

Description of staff screen flow in json:

```
{ "diagram_name": "Staff Screen Flow Diagram", "objects": [ { "object_name": "Login", "type": "screen" }, { "object_name": "StaffDashboard", "type": "screen" }, { "object_name": "RequestList", "type": "screen" }, { "object_name": "SendRequest", "type": "screen" } ], "connections": [ { "source": "Login", "target": "StaffDashboard", "action": "Login successful" }, { "source": "StaffDashboard", "target": "RequestList", "action": "View request list" }, { "source": "StaffDashboard", "target": "SendRequest", "action": "Send new request" }, { "source": "RequestList", "target": "StaffDashboard", "action": "Back" }, { "source": "SendRequest", "target": "StaffDashboard", "action": "Back" } ] }
```

Table 3: Activity diagram of Staff description

#	Feature	Screen	Description
1	Login	Login Screen	Allows staff members to enter their credentials (username/password) to access the staff portal.
2	Dashboard	Dashboard Screen	Main landing page after login; provides quick access to “Send Request” and “Request List” functionality.
3	Send Request	Send request	Staff can fill out and submit a new request.
4	View Request List	Request list (staff side)	Displays a list of all requests created by the staff member, including status and basic details.

c) Manager screen flow

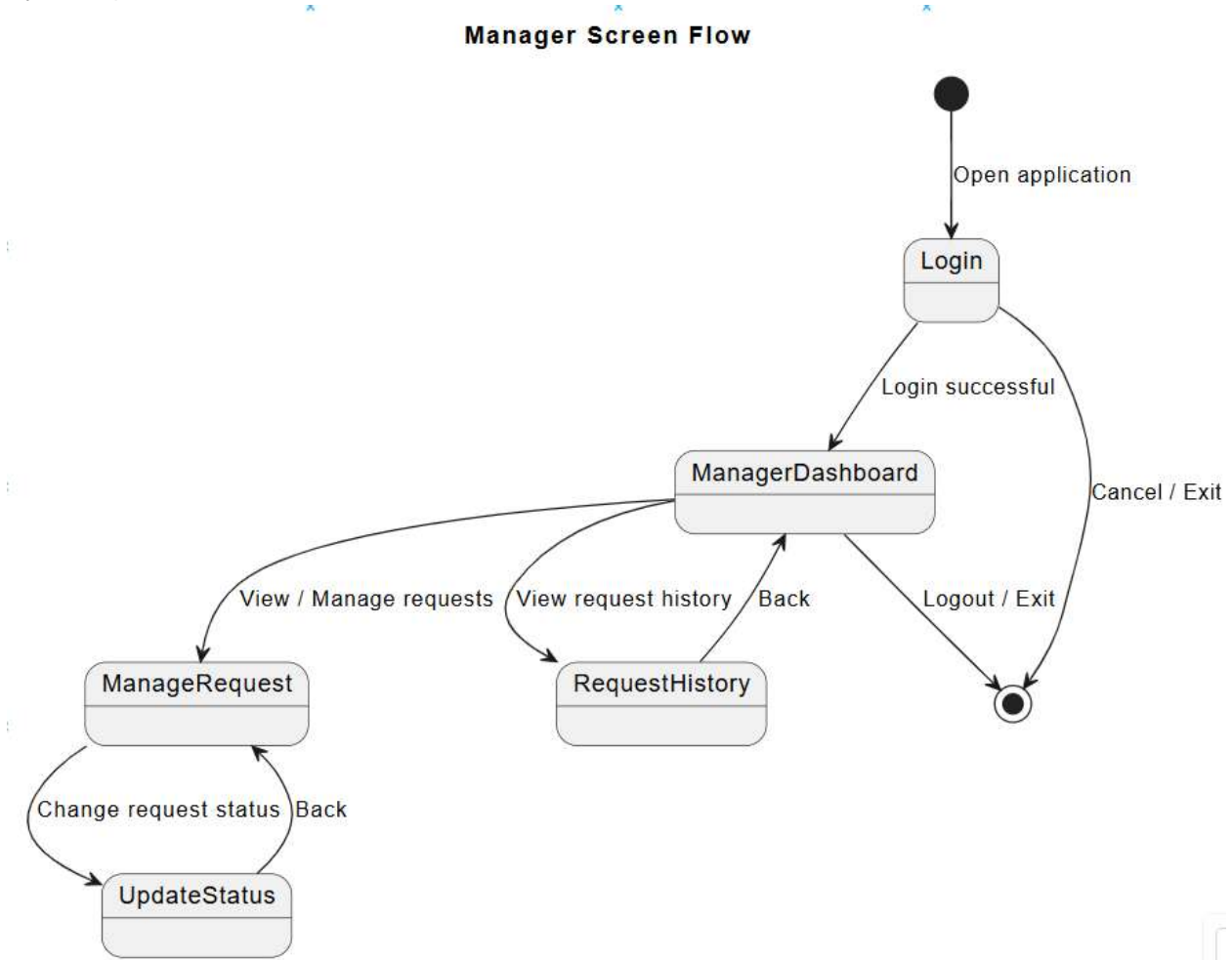


Figure 3: Manager Screen Flow diagram

- **Purpose:** To illustrate the screen flow specifically for the Manager role.
- **Elements:**
 - **Start Point:** Black circle
 - **End Point:** Black circle with a white dot in the center
 - **Screens:** Login, ManagerDashboard, ManageRequest, RequestHistory, UpdateStatus
 - **Actions:** Open application, Login successful, View / Manage requests, View request history, Back, Change request status, Logout / Exit, Cancel / Exit
- **Flow:**
 1. User opens the application.
 2. User logs in.
 3. Upon successful login, user is taken to the ManagerDashboard.

4. From the ManagerDashboard, the user can choose to "View / Manage requests" or "View request history".
5. Within "Manage requests", the user can "Change request status", leading to the UpdateStatus screen.
6. The user can navigate back to the ManagerDashboard from either ManageRequest or RequestHistory.
7. The user can "Logout / Exit" or "Cancel / Exit" the application.

Description of Manager screen flow in json:

```
{ "diagram_name": "Manager Screen Flow Diagram", "objects": [ { "object_name": "Login", "type": "screen" }, { "object_name": "ManagerDashboard", "type": "screen" }, { "object_name": "ManageRequest", "type": "screen" }, { "object_name": "RequestHistory", "type": "screen" }, { "object_name": "UpdateStatus", "type": "screen" } ], "connections": [ { "source": "Login", "target": "ManagerDashboard", "action": "Login successful" }, { "source": "ManagerDashboard", "target": "ManageRequest", "action": "View / Manage requests" }, { "source": "ManagerDashboard", "target": "RequestHistory", "action": "View request history" }, { "source": "ManageRequest", "target": "UpdateStatus", "action": "Change request status" }, { "source": "ManageRequest", "target": "ManagerDashboard", "action": "Back" }, { "source": "RequestHistory", "target": "ManagerDashboard", "action": "Back" } ] }
```

2.2 Screen Descriptions

Table 4: Manager activity diagram descriptions

#	Feature	Screen	Description
1	Login	Login Screen	Allows managers to enter their credentials (username/password) to access the manager portal.
2	Dashboard	Dashboard Screen	Main landing page after login; provides quick access to “Manage Request,” “Send Request,” and “Request List.”
3	Manage Request	Manage request	Allows the manager to view and manage incoming requests
4	View Request History	View Request History	Displays a list of all staff’s requests
5	Update Request	Update Request	Lets the manager change the status of a request (Approved, Rejected, Pending)

2.3 Screen Authorization

Table 5: Screen authorization

Screen	Staff	Manager
Request	X	
Dashboard	X	X
Request List	X	
Request History		X
Manage Request		X

2.4 State transition diagram of Request:

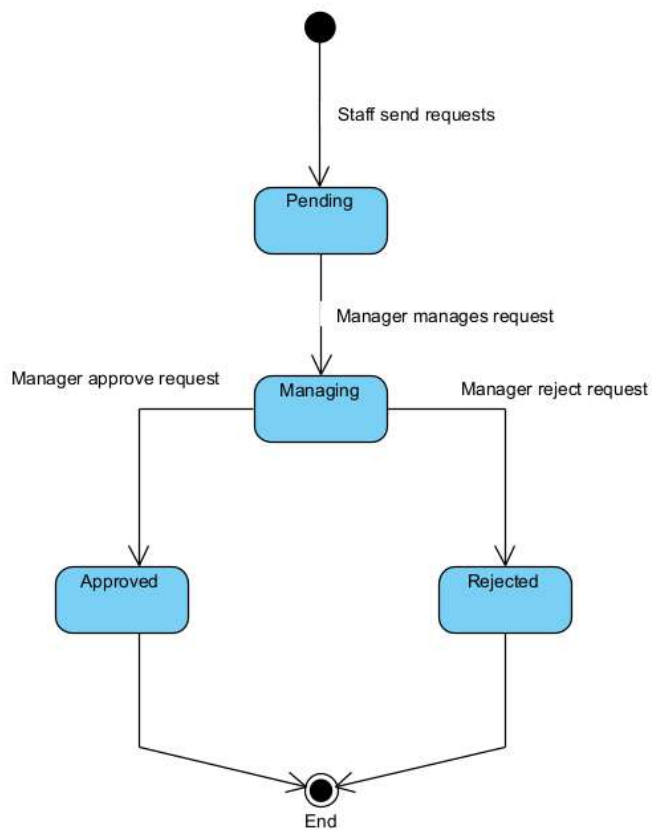


Figure 4: State transition diagram of Request

Purpose:

This diagram illustrates the lifecycle of a request in the system, from creation, processing, approval, or rejection, to completion.

Elements:

- **States:** Pending, Processing, Approved, Rejected, Closed.

- **State Transitions:**

- Pending → Processing: When the manager starts reviewing the request.
- Processing → Approved: When the manager approves the request.
- Processing → Rejected: When the manager rejects the request.
- Approved → Closed: When the request is fulfilled.
- Rejected → Closed: When the request is finalized and no further processing is needed.

Flow:

1. **The staff** creates a new request → the state transitions to Pending.
2. **The manager** reviews the request → the state transitions to Processing.
3. **The manager makes a decision:**
 - If approved, the state transitions to Approved.
 - If rejected, the state transitions to Rejected.
4. **When the request is completed:**
 - If Approved, the state transitions to Closed.
 - If Rejected, the state transitions to Closed.
5. The system completes the request lifecycle when the Closed state is reached.

Description of state transition diagram of request in json:

```
{ "diagram_name": "Request State Transition Diagram", "objects": [ { "object_name": "Pending", "type": "state" }, { "object_name": "Processing", "type": "state" }, { "object_name": "Approved", "type": "state" }, { "object_name": "Rejected", "type": "state" }, { "object_name": "Closed", "type": "state" } ], "connections": [ { "source": "Pending", "target": "Processing", "action": "Manager starts reviewing" }, { "source": "Processing", "target": "Approved", "action": "Manager approves request" }, { "source": "Processing", "target": "Rejected", "action": "Manager rejects request" }, { "source": "Approved", "target": "Closed", "action": "Request is fulfilled" }, { "source": "Rejected", "target": "Closed", "action": "Request is finalized" } ] }
```

3. System High Level Design

3.1 Database Schema:

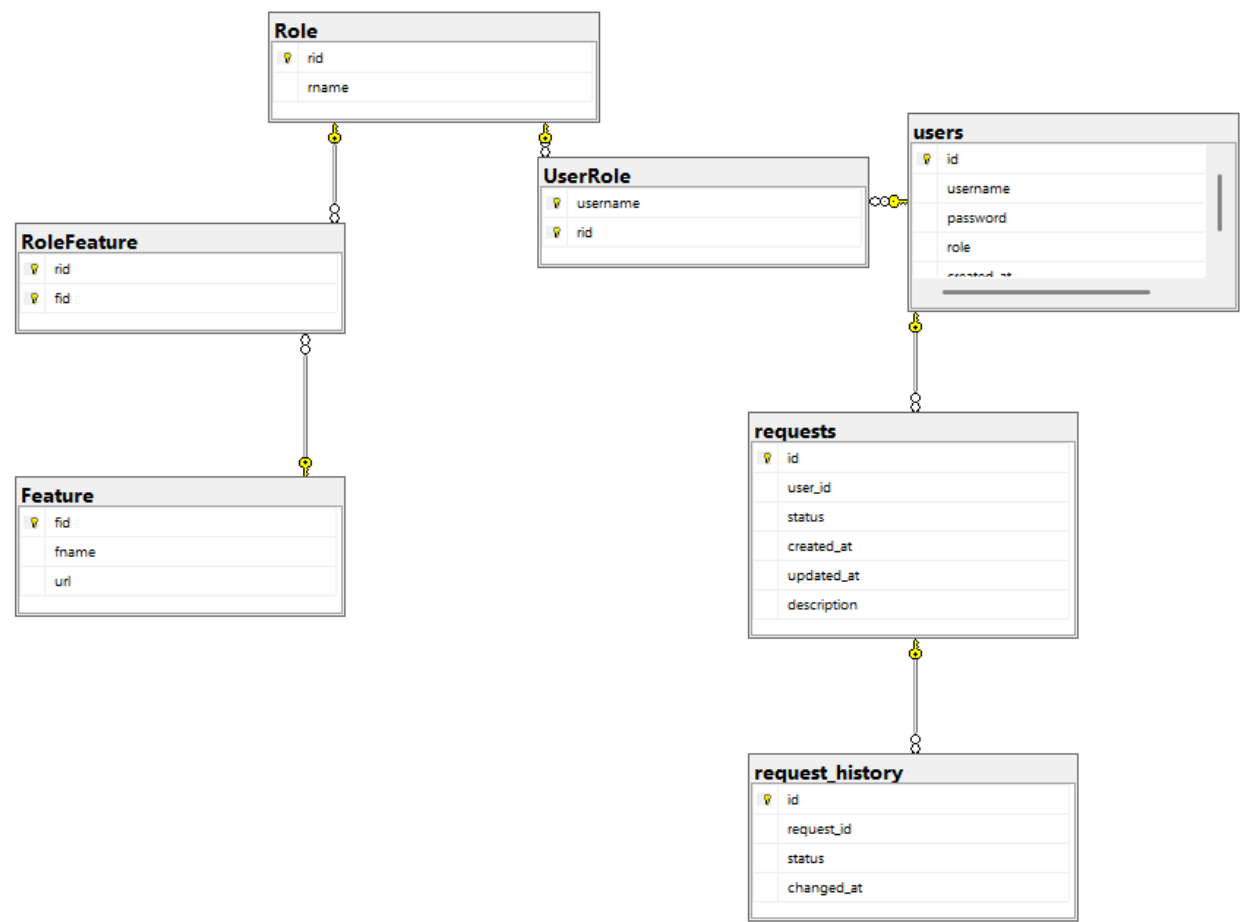


Figure 5: Database schema

3.2 Description:

Table 6: ERD diagram description

No	Table	Description
01	requests	id (INT, IDENTITY, PRIMARY KEY) user_id (INT, NOT NULL) status (VARCHAR(50), NOT NULL) created_at (DATETIME, NULL) updated_at (DATETIME, NULL) description (NVARCHAR(1000), NULL)

02	request_history	id (INT, IDENTITY, PRIMARY KEY) request_id (INT, NOT NULL) status (VARCHAR(50), NOT NULL) changed_at (DATETIME, NULL)
03	users	id (INT, IDENTITY, PRIMARY KEY) username (VARCHAR(50), NOT NULL, UNIQUE) password (VARCHAR(255), NOT NULL) role (INT, NULL) created_at (DATETIME, NULL)
04	UserRole	username (VARCHAR(50), NOT NULL) rid (INT, NOT NULL) PRIMARY KEY (username, rid)
05	Role	rid (INT, IDENTITY, PRIMARY KEY) rname (VARCHAR(50), NOT NULL)
06	RoleFeature	rid (INT, NOT NULL) fid (INT, NOT NULL): PRIMARY KEY (rid, fid)
07	Feature	fid (INT, IDENTITY, PRIMARY KEY) fname (VARCHAR(50), NOT NULL) url (VARCHAR(100), NOT NULL)

Description of ERD in json:

```
{ "diagram_name": "ERD Diagram", "objects": [ { "object_name": "Role", "type": "table", "columns": [ "rid", "rname" ] }, { "object_name": "RoleFeature", "type": "table", "columns": [ "rid", "fid" ] }, { "object_name": "Feature", "type": "table", "columns": [ "fid", "fname", "url" ] }, { "object_name": "UserRole", "type": "table", "columns": [ "username", "rid" ] }, { "object_name": "users", "type": "table", "columns": [ "id", "username", "password", "role" ] }, { "object_name": "requests", "type": "table", "columns": [ "id", "user_id", "status", "created_at", "updated_at", "description" ] },
```

```
{ "object_name": "request_history", "type": "table", "columns": ["id", "request_id", "status",
"changed_at"] }, "connections": [ { "source": "Role", "target": "RoleFeature", "type": "one-to-many" }, { "source": "Feature", "target": "RoleFeature", "type": "one-to-many" }, { "source": "Role",
"target": "UserRole", "type": "one-to-many" }, { "source": "users", "target": "UserRole", "type": "one-to-many" }, { "source": "users", "target": "requests", "type": "one-to-many" }, { "source": "requests",
"target": "request_history", "type": "one-to-many" } ] }
```

II. Requirement Specifications

1. Use Case Description

1.1 Use Case: Staff Send Request

a. Diagram(s) Staff

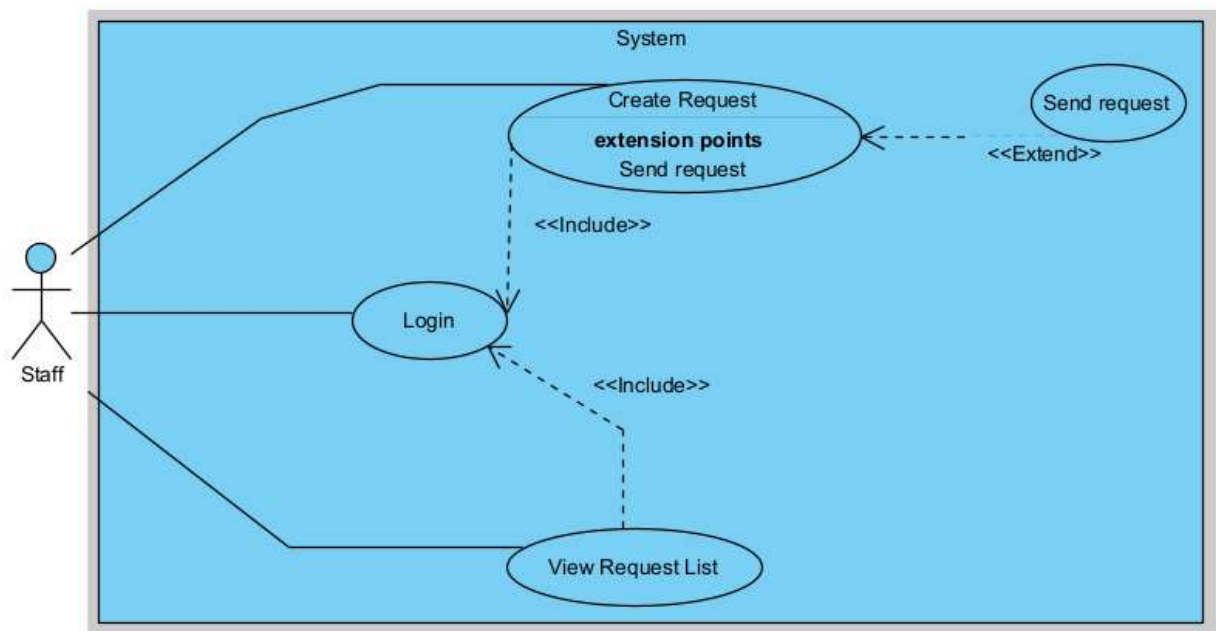


Figure 6: Staff use case

- **Purpose:** To illustrate the use cases (functionalities) available to the Staff role.
- **Elements:**
 - **Actor:** Staff (represented by a stick figure)
 - **Use Cases:** Login, Send Request, View Request List
 - **Relationships:** Lines connecting the actor to use cases indicate which actions the actor can perform.

- **Include Relationship:** The "Send Request" and "View Request List" use cases both include the "Login" use case, indicating that login is required for those actions.
- **Extend Relationship:** The "Send request" use case extends from the "Create Request" use case, indicating that sending a request is a specific way to create a request.
- **Flow:** The diagram doesn't explicitly show a flow, but it implies that the Staff actor must first log in before performing other actions like sending or viewing requests.

Description of staff use case diagram in json:

```
{ "diagram_name": "Staff Use Case Diagram", "objects": [ { "object_name": "Staff", "type": "actor" }, { "object_name": "Login", "type": "use_case" }, { "object_name": "Send Request", "type": "use_case" }, { "object_name": "View Request List", "type": "use_case" } ], "connections": [ { "source": "Staff", "target": "Send Request", "type": "participates" }, { "source": "Staff", "target": "View Request List", "type": "participates" }, { "source": "Send Request", "target": "Login", "type": "includes" }, { "source": "View Request List", "target": "Login", "type": "includes" } ] }
```

b. Descriptions

Table 7: Staff use case description

#	Feature	Screen	Description
1	Staff	Login	Staff login with valid username, valid password to login to the system
2		Request	Staff can create a new request and send to the manager
3		Request List	Staff can view all request are sent to the manager, to see the status is approved, rejected, pending

Table 8: Use case description of UC-SR001: Send Request

UC ID and Name:	Use Case ID: UC-SR001 Use Case Name: Send Request		
Primary Actor:	Staff	Secondary Actors:	Manager, System
Trigger:	The Staff user selects "Send Request" from the Staff Dashboard to create a new request send to the manager		

Description:	This use case allows a Staff user to create and submit a new request (e.g., for approval, service, or any organizational need). After entering the receiver's name and all required information, the system validates the request and stores it. The user is then notified of the successful creation of the request.	
Preconditions:	The Staff user is authenticated and has a valid session (already logged in). (Screen: Login) The Staff user has permission to create a new request.	
Postconditions:	A new request record is stored in the system database. The Staff user can view the newly created request in their "Request List." After the requests sent successfully, they will redirect to UC-SR002 to let the staff see all their requests. Besides, it also redirect to UC-MR001 and UC-MR002 to let the manager can watch and manage their requests	
Normal Flow:	1. Staff navigates to the Staff Dashboard and chooses "Send Request." 2. System displays the Send Request form. 3. Staff enters receiver's name and the request details and clicks Submit. 4. If the staff want to cancel the process send request, click "Cancel" or if the staff want to successfully send request, click "Ok"	3.1 System validates the input data (e.g., required fields, format). 3.2 System saves the request in the database. 3.3 System alert a message to confirm with the content: "Are you sure?"
Alternative Flows:	Invalid Data Staff submits the form with missing or incorrect data. System displays an error message Staff corrects the data and resubmits.	
Exceptions:	Session Timeout: If the Staff's session expires, they are prompted to log in again before proceeding.	
Priority:	High (Must Have)	
Frequency of Use:	This action can occur several times per day per Staff user.	

Business Rules:	BR1 BR2 BR3 BR4
Use Case related	Related Use Cases: Related Use Cases: “View Request History” (UC-MR002) , “View Request List” (UC-SR002), “Manage Request (UC-MR001)
Screen related	Screen: Login -> Dashboard (Staff) -> Send Request
Assumptions:	The authentication process is handled separately and is already complete when entering this use case. The Staff user has been assigned the role that permits request creation.

**Business Rules of use case Staff: Send Request:*

Provide the business rules those are applied only to the use case

Table 9: Business rule of use case UC-SR001:Send Request

ID	Business Rule	Business Rule Description
BR1	Unique Identifier	The request ID must be a unique, system-generated number with a length of 4 characters. It should not be editable by the user. Constraint: Ensure uniqueness through automated testing and database checks. Verify length during unit and integration tests
BR2	Mandatory Fields	The "Description" field is mandatory. Constraint: Field should be marked with an asterisk (*) or visual indicator. Form submission should be blocked if the field is empty. Automated tests should verify this behavior002E
BR3	Description Length	The maximum character limit for the "Description" field is 1000 characters. Constraint: Input field should prevent entry beyond 1000 characters. Display a character count to the user. Test cases should check for truncation or error messages upon exceeding the limit.
BR4	Default Status	New requests should default to a "Pending" status. Constraint: Verify the initial status in automated tests after request creation. Ensure that the database correctly stores the default status.
BR5	Valid Recipient	The "To:" [field] must contain a valid recipient's name. Ensure that the input is validated against a predefined list of users. Form submission should be blocked if an invalid or empty recipient is provided.

1.2 Use Case: View Request List

Table 10: Use case description of UC-SR002: View Request List

UC ID and Name:	Use Case ID: UC-SR002 Use Case Name: View Request List		
Primary Actor:	Staff	Secondary Actors:	System
Trigger:	The Staff user selects “Request List” from the Staff Dashboard to view all requests they have created.		
Description:	This use case allows a Staff user to view all of their previously created requests, along with relevant details such as request titles, submission dates, and current statuses. The user can then decide to review, update, or track the progress of any specific request.		
Preconditions:	The Staff user is authenticated (logged in). (Screen: Login) The Staff user has the role/permission to view their own requests.		
Postconditions:	The Staff user has access to an up-to-date list of their requests. The system displays each request’s essential details		
Normal Flow:	Staff navigates to the Staff Dashboard. Staff clicks “Request List”. Staff can see the status of request is approved, rejected or pending	System displays the Request List page, showing each request’s title, status, and other details. 3.1 System retrieves the Staff’s requests from the database.	
Alternative Flows:			
Exceptions:	Session Timeout: If the Staff’s session expires, they are prompted to log in again before proceeding.		
Priority:	High (Must Have)		
Frequency of Use:	This action can occur several times per day per Staff user.		
Business Rules:	BR1 BR2 BR3		
Use Case related:	Related Use Cases: “Send Request” (UC-SR001), “Manage Request” (Manager updates the request status).		

Screen related:	Screen: Login -> Dashboard (Staff) -> View Request History
Assumptions:	The authentication process is handled separately and is already complete when entering this use case. The Staff user has been assigned the role that permits request creation.

**Business Rules of use case Staff: View Request List:*

Provide the business rules those are applied only to the use case

Table 11 Business rule of use case: UC-SR002: View Request List

ID	Business Rule	Business Rule Description
BR1	Display of Identifier and Status	Each request entry must display its unique identifier and current status. Constraint: UI tests should verify the presence and correctness of these elements for every listed request
BR2	Authorization	Staff can only view their own requests. Managers can view all requests. Constraint: Implement role-based access control. Test with both staff and manager accounts to ensure proper data filtering
BR3	Finalized Status	Once a manager has either approved or rejected a pending request, its status cannot be changed.

1.3 Use Case: Manager Manage Request

a. Diagram(s) Manager:

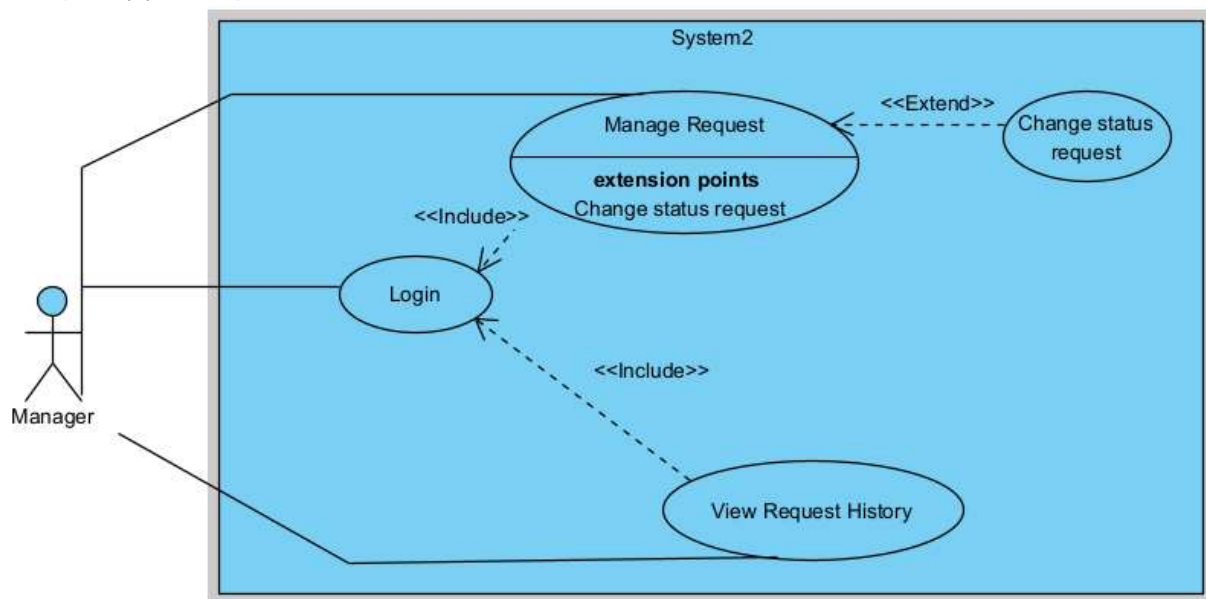


Figure 7: Manager Use Case diagram

Purpose: To illustrate the use cases (functionalities) available to the Manager role.

Elements:

- **Actor:** Manager (represented by a stick figure)
- **Use Cases:** Login, Manage Request, View Request History
- **Relationships:** Lines connecting the actor to use cases indicate which actions the actor can perform.
- **Include Relationship:** The "Manage Request" and "View Request History" use cases both include the "Login" use case, indicating that login is required for those actions.
- **Extend Relationship:** The "Change status request" use case extends from the "Manage Request" use case, indicating that changing the status is a specific way to manage a request.

Flow: The diagram doesn't explicitly show a flow, but it implies that the Manager actor must first log in before performing other actions like managing requests or viewing request history.

Description of manager use case diagram in json:

```
{ "diagram_name": "Manager Use Case Diagram", "objects": [ { "object_name": "Manager", "type": "actor" }, { "object_name": "Login", "type": "use_case" }, { "object_name": "Manage Request", "type": "use_case" }, { "object_name": "View Request History", "type": "use_case" } ], "connections": [ { "source": "Manager", "target": "Manage Request", "type": "participates" }, { "source": "Manager", "target": "View Request History", "type": "participates" }, { "source": "Manage Request", "target": "Login", "type": "includes" }, { "source": "View Request History", "target": "Login", "type": "includes" } ] }
```

Figure 1: Manager Use case

b. Descriptions

Table 12: Manager use case description

ID	Actor	Use Case	Description
1	Manager	Login	Manager login with valid username, valid password to login to the system
2		Manage Request	Manager can adjust the status of Staff's request to approved or rejected status
3		Request History	Manager can see all the request are sent by Staff with their status

Table 13: Use case description of UC-MR001: Manage Request

UC ID and Name:	Use Case ID: UC-MR001 Use Case Name: Manage Request		
Primary Actor:	Manager	Secondary Actors:	Staff, System
Trigger:	The Manager selects “Manage Request” from the Manager Dashboard to view and update staff requests.		
Description:	This use case allows a Manager to view, edit, or update staff-submitted requests. Actions may include changing request status. The system ensures any modifications are validated and saved.		
Preconditions:	The Manager is authenticated and logged into the system. (Screen: Login) The Manager has permissions to manage staff requests (e.g., update statuses). The requests must be sent successfully by the Staff before the Manager check manage request (Screen: Send Request)		
Postconditions:	The request is updated in the database to Approved or Rejected		
Normal Flow:	Manager opens the Manager Dashboard. Manager clicks “Manage Request.” Manager click button “Approved” or “Rejected”	System retrieves a list of pending or all requests from the database. System alert “Are you sure you want to approve this request?” when the manager clicks the Approved button and clicks yes in the alert, the status of request changes to “Approved”. System alert “Are you sure you want to reject this request?” when the manager clicks the Approved button click yes, the status of request changes to “Rejected”.	
Alternative Flows:	Cancel Update Manager decides not to proceed with changes and clicks Cancel. System discards any unsaved modifications and returns the Manager to the Manage Request list.		
Exceptions:	Session Timeout: If the Manager’s session expires, they are prompted to log in again before proceeding.		
Priority:	High (Must Have)		

Frequency of Use:	This action can occur several times per day
Business Rules:	BR1 BR2 BR3
Use Case related:	Related Use Cases: “View Request History” (UC-MR002) , “View Request List” (UC-SR002), “Send Request”(UC-SR001)
Screen related:	Screen: Login -> Dashboard (Manager) -> Manage Request
Assumptions:	The authentication process is handled separately and is already complete when entering this use case. The Manager user has been assigned the role that permits request creation.

**Business Rules of use case Manage Request:*

Provide the business rules those are applied only to the use case

Table 14: Business rule of use case: UC-MR001: Manage Request

ID	Business Rule	Business Rule Description
BR1	Finalized Status	Once a manager has either approved or rejected a pending request, its status cannot be changed.
BR2	Timestamp and User ID	Each status change must have a timestamp and the ID of the user who made the change. Constraint: Database should store these values. UI should display them accurately. Tests should verify the presence and correctness of timestamp and user ID information
BR3	Read-Only History	Historical records are read-only. Managers cannot alter past entries. Constraint: Disable editing functionality for historical data. Attempt to modify data through the UI and API to confirm that changes are not persisted.

1.4 Use Case: Manager View Request History

Table 15: use case description of UC-MR001: View Request History

UC ID and Name:	Use Case ID: UC-MR002 Use Case Name: View Request History
-----------------	--

Primary Actor:	Manager	Secondary Actors:	System
Trigger:	The Manager selects “Request History” (or a similar option) from the Manager Dashboard or from a detailed request view.		
Description:	This use case allows a Manager to view the history of requests, including changes in status, timestamps, and possibly who made each change. The Manager can analyze trends or confirm that procedures were followed properly.		
Preconditions:	Manager is logged in with valid credentials. (Screen: Login) Manager has permission to access historical records		
Postconditions:	The Manager can view an up-to-date history of requests, including all status changes		
Normal Flow:	Manager opens the Manager Dashboard. Manager selects “View Request History.” Manager returns to the Manager Dashboard or closes the history screen.	System displays the Request History screen, showing request’s status changes System retrieves historical data (e.g., all requests or a subset, depending on the design).	
Alternative Flows:	Manager closes the history view and returns to the Manage Request screen or dashboard.		
Exceptions:	Session Timeout: If the Manager’s session expires, they are prompted to log in again before proceeding.		
Priority:	Medium		
Frequency of Use:	Possibly periodic		
Business Rules:	BR1 BR2 BR3		
Related Use Cases:	“Manage Request (UC-MR001), “Send Request”(UC-SR001)		
Screen related:	Screen: Login -> Dashboard (Manager) -> View Request History		
Assumptions:	The authentication process is handled separately and is already complete when entering this use case.		

	The Manager user has been assigned the role that permits request creation.
--	--

**Business Rules of Manager feature:*

2. Common Functions

2.1 Use Case: Login System

a. Functional Description

Table 16: use case description of UC_1: Login

UC ID and Name:	Use Case ID: UC_1 Use Case Name: Login		
Primary Actor:	Staff, Manager	Secondary Actors:	None
Trigger:	A user initiates the login process by entering credentials on the login screen.		
Description:	This use case describes the process of a registered user logging into the system using valid credentials. The system verifies the credentials and grants access to the appropriate dashboard based on the user's role.		
Preconditions:	The user must be a registered and active system user. The system must be online and able to process authentication requests. The user must have valid login credentials.		
Postconditions:	If authentication is successful, the user gains access to their respective dashboard. If authentication fails, the system logs the failed attempt and notifies the user.		
Normal Flow	1. The user accesses the login screen. 2. The user enters their username and password.	2.1 The system verifies the credentials against the database. 2.2 If the credentials are valid: The system grants access and redirects the user to their dashboard. The system logs the successful login.	
Alternative Flows:	None		

Exceptions:	Incorrect Credentials If the user enters invalid credentials, the system displays an error message and allows the user to retry
Priority:	High
Frequency of Use:	Multiple times per day per user
Business Rules:	BR1
Use case related	Use case: “Manage Request (UC-MR001), “Send Request”(UC-SR001), “View Request History” (UC-MR002) , “View Request List” (UC-SR002)
Screen related	Screen: Login
Assumptions:	Users will enter valid credentials. The system’s authentication server is operational.

b. Business Rules

Table 17: Business rule of use case UC_1: Login

ID	Business Rule	Business Rule Description
BR1	Login success	User login with their account, the account must match with account data in the database

III. Design Specifications

1. Screen design:

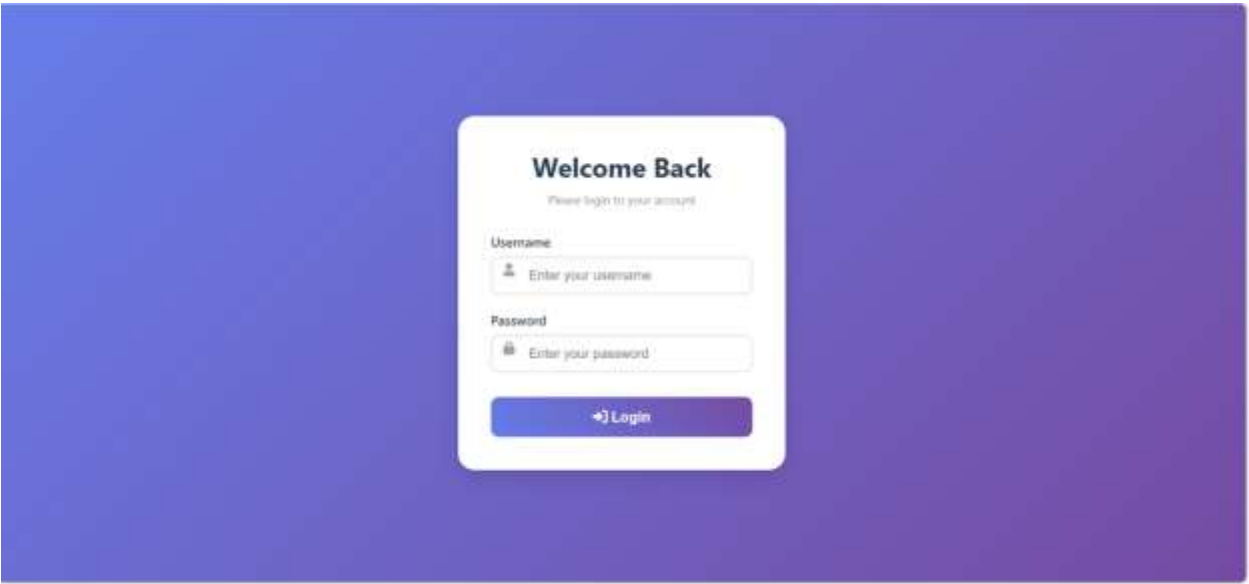
1.1 Screen: Login

This screen allows user to be authenticated to the system screens/functionalities.

Related use cases:

- UC_1: Login

UI Design



Screen 1: Screen Login

Table 18: Element description of Login screen

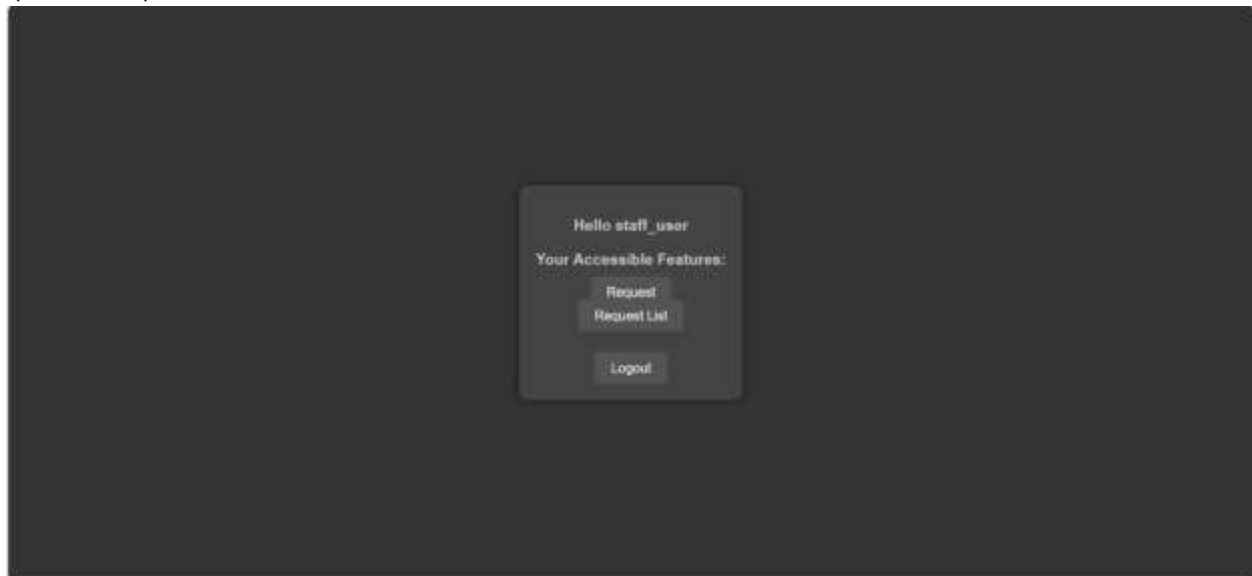
Element name	HTML Tag	Purpose	Description
Login Header	<div class="login-header"><h2>Welcome Back</h2><p>Please login to your account</p></div>	Page heading for login	Provides context for the page, informing the user that they need to log in to access their account.
Username Input Field	<input id="username" name="username" type="text"/>	Allows the user to input their username	This field allows the user to enter their username, identifying the account they wish to log into.
Password Input Field	<input id="password" name="password" type="password"/>	Allows the user to input their password	This field allows the user to enter their password to authenticate their account.

Login Button	<code><button type="submit" class="login-btn"></code>	Submits the login form	This button performs the primary action of the page: sending the username and password for authentication.
Error Message	<code><div class="error-message"></code>	Displays an error message when login information is incorrect	If the login information is incorrect, this message will appear, allowing the user to correct their input.

1.2 Screen: Dashboard

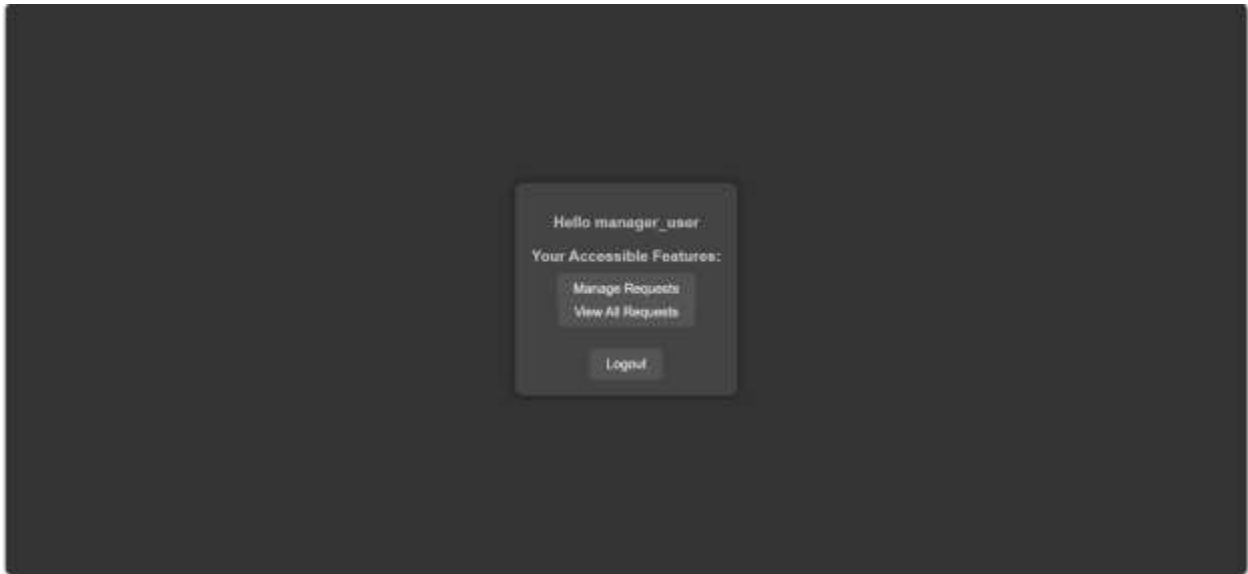
UI Design

(Staff-side)



Screen 2: Dashboard (Staff)

(Manager-side)



Screen 3: Dashboard (Manager)

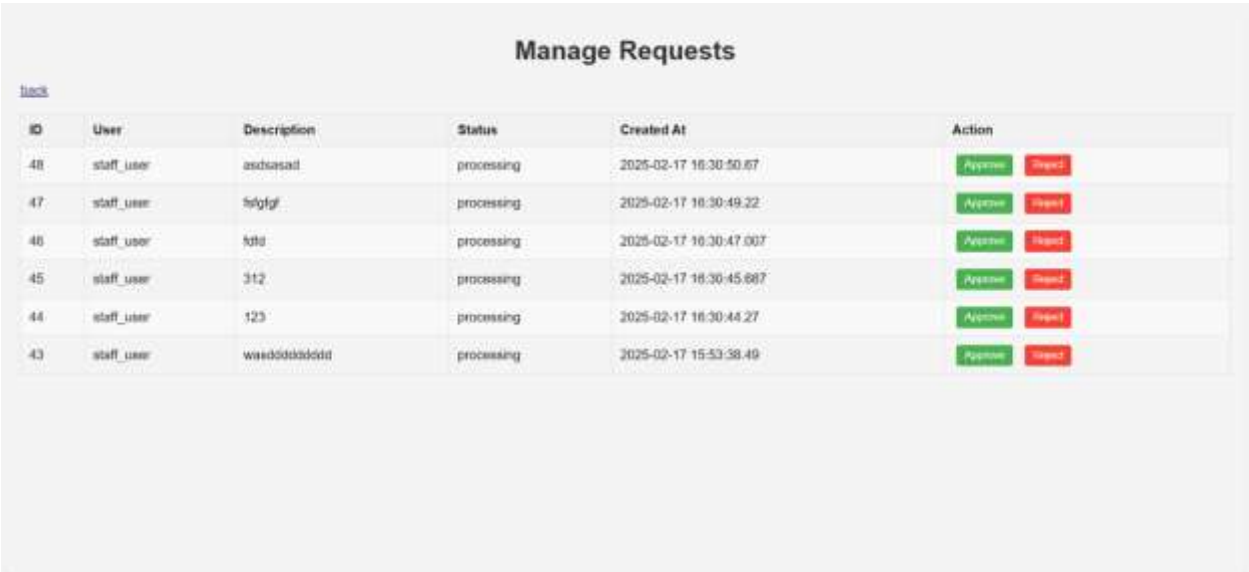
Table 19: Element description of Dashboard screen

Element name	HTML Tag	Purpose	Description
Features List Title	Your Accessible Features:	Indicate the section showing available features.	Inform the manager of the features they have access to based on their role.
Feature Links List	<ul class="feature-links">	Display clickable links for managers' features.	Lists the links corresponding to features.

Feature Link	\${feature.name}	Link to each accessible feature.	Links to features that r can be accessed by each role.
Logout Link	Logout	Provide a logout option for the user.	Allows <sb> to log out of the application.

1.3 Screen: Manager side: Manage Request:
Use case related: UC-MR001: Manage Request

UI Design



Screen 4: Manage Request (Manager)

Table 20: Element description of Manage Request screen

Element name	HTML Tag	Purpose	Description
--------------	----------	---------	-------------

Page Title	<code><h1>Manage Requests</h1></code>	Display the title of the page.	Indicates the function of this page: to manage and process requests.
Requests Table	<code><table></code>	Display a list of requests to manage.	The main table that shows each request's ID, user, description, status, and creation date.
Table Header (ID)	<code><th></code>	Label each column in the table.	Defines the columns in the table: ID, User, Description, Status, Created At, Action.
Request Row	<code><tr></code>	Display individual requests in the table.	For each request, a row is created with details such as ID, User, Description, Status, Created At.
Action Buttons	<code><form><button></button></form></code>	Enable actions (approve or reject) on each request.	Provides forms for the manager to approve or reject requests. The action buttons are displayed based on the request's status.
Approve Button	<code><button class="approve"></code>	Approve a request.	A button that allows the manager to approve the request. The button is only visible if the request is not already approved or rejected.
Action Buttons	<code><button class="reject"></code>	Reject a request.	A button that allows the manager to reject the request. The button is only visible if the request is not already approved or rejected.

Empty State Message	<td colspan="6">	Display message when there are no pending requests.	If no requests are available, a message "No pending requests" is shown across the entire table.
Confirmation Popup	<script>	Confirm the action of approving or rejecting.	A JavaScript function to show a confirmation prompt before proceeding with approval or rejection actions.

1.4 Screen: Manager side: View Request History:
Use case related: UC-MR002:View Request History

UI Design

All Requests					
Back					
Request ID	User ID	Description	Status	Created At	Updated At
48	staff_user	asdasad	processing	2025-02-17 16:30:50.67	2025-02-17 16:30:50.67
47	staff_user	fdfgfd	processing	2025-02-17 16:30:49.22	2025-02-17 16:30:49.22
46	staff_user	fdfd	processing	2025-02-17 16:30:47.007	2025-02-17 16:30:47.007
45	staff_user	312	processing	2025-02-17 16:30:45.667	2025-02-17 16:30:45.667
44	staff_user	123	processing	2025-02-17 16:30:44.27	2025-02-17 16:30:44.27
43	staff_user	wasddddddddd	processing	2025-02-17 15:53:36.49	2025-02-17 15:53:36.49
42	staff_user	123ttas	approved	2025-01-17 08:39:34.16	2025-02-17 15:50:06.07
41	staff_user	Okio Okio	approved	2025-01-16 13:48:09.14	2025-01-16 13:48:37.76
40	staff_user	Hello guys	rejected	2025-01-16 13:30:28.93	2025-01-16 13:41:41.577
33	staff_user	hello	approved	2025-01-14 11:09:03.78	2025-01-14 21:29:13.723
32	staff_user	Marked as completed successfully	approved	2025-01-14 10:24:50.6	2025-01-14 21:29:14.27
25	staff_user	General description to be updated	approved	2025-01-14 10:17:15.07	2025-01-14 21:29:15.493
24	staff_user	Test description	rejected	2025-01-14 10:15:03.0	2025-01-16 13:41:43.403
20	staff_user	Task successfully approved	rejected	2025-01-14 10:08:56.43	2025-01-16 13:41:44.707

Screen 5: Request History(Manager)

Table 21: Element description of View Request History screen

Element name	HTML Tag	Purpose	Description
--------------	----------	---------	-------------

Page Title	All Requests	Display the title of the page.	Indicates the function of this page: to show all requests in the system, including their status.
Requests Table	<table>	Display a list of all requests.	The main table showing all request details: ID, User ID, Description, Status, Created At, Updated At.
Table Header	<th>	Label each column in the table.	Defines the columns: Request ID, User ID, Description, Status, Created At, Updated At.
Request Row	<tr>	Display individual requests in the table.	Each request is displayed in a row with the respective details such as ID, User, Description, etc.
Status Classes	<tr class="{request.status}">	Display color coding based on request status.	Dynamically applies a class (approved or rejected) to change the row color based on the request's status.
Approved Row	Approved Row	Highlight approved requests.	If the request status is "approved", it will be highlighted with a green background.
Rejected Row	<tr class="rejected">	Highlight rejected requests.	If the request status is "rejected", it will be highlighted with a red background.
Request Data	Request Data	Request Data	Each cell displays the specific information for the request: ID, username, description, status, etc.

1.5 Screen: Staff side: Send Request:

Use case related: UC-SR001: Send Request

UI Design

[Back to Dashboard](#)

Create New Request

From: staff_user

To:

Description:

Submit Request

Screen 6:Send Request (Staff)

Table 22: Element description of Send Request screen

Element name	HTML Tag	Purpose	Description
Page Title	Create New Request	Display the title of the page.	Indicates the function of the page: creating a new request.
Hidden User ID Input	<input id="userId" name="userId" type="hidden" value="{session.user.getUserId}">	Store user ID without displaying it.	The user's ID is stored in a hidden field and passed with the form when submitting the request.
Description Label	<label for="description">Description:</label>	Label for the description input field.	DProvides context for the input field where the user will

			describe their request.
Description Textarea	<code><textarea id="description" name="description" rows="4" cols="50" required></textarea></code>	Input field for the description of the request.	A text area for the user to enter the details of their request. It is marked as required.
Input receiver	<code><input id="approveMan" name="approveMan" type="text" value=""></code>	Input Field for request Recipient's name	This input field is used to enter the name of the person who will receive the request
Submit Button	<code><button type="submit" onsubmit="showAlert()">Submit Request</button></code>	Submit the form to create a new request.	A button to submit the request form. It triggers the showAlert() function upon submission.
Alert Script	<code><script> function showAlert() { alert("Are you sure?"); } </script></code>	Display an alert after submission.	A JavaScript function to show an alert notifying the staff to let they know that do they want to send success or not

1.6 Screen: Staff side: Request List:

Use case related: UC-SR002: View Request List

UI Design

Request List			
back			
ID	Description	Status	Created Date
17	Action not approved	rejected	2025-01-14 00:45:01.88
19	Process successfully completed	approved	2025-01-14 08:44:05.91
20	Task successfully approved	rejected	2025-01-14 10:08:56.43
24	Test description	rejected	2025-01-14 10:15:03.0
25	General description to be updated	approved	2025-01-14 10:17:15.07
32	Marked as completed successfully	approved	2025-01-14 10:24:50.6
33	hello	approved	2025-01-14 11:00:03.70
40	Hello guys	rejected	2025-01-16 13:30:28.93
41	Oke Oke	approved	2025-01-16 13:48:00.14
42	1231las	approved	2025-01-17 08:39:34.16
43	wassdtdtdtdtd	processing	2025-02-17 15:53:38.49
44	123	processing	2025-02-17 16:30:44.27
45	312	processing	2025-02-17 16:30:45.687
46	456	processing	2025-02-17 16:30:46.000

Screen 7: Request List (Staff)

Table 23: Element description of View Request List screen

Element name	HTML Tag	Purpose	Description
Page Title	Request List	Display the title of the page.	Indicates the function of the page: viewing the list of requests.
Table Header	<thead><tr><th>ID</th><th>Description</th><th>Status</th><th>Created Date</th></tr></thead>	Header for the request list table.	Defines the columns for request ID, description, status, and creation date.

Table Row	<c:forEach var="request" items="\${requests}">	Display each request in the table.	Loops through all requests in the list and displays the corresponding information in the table rows.
Row Class (Status-based)	<tr class="\${request.status}">	Change the row background color based on request status.	Applies a background color to each row based on the request's status (approved, rejected, pending, etc.).
Request ID	<td>\${request.id}</td>	Display the request ID.	Displays the unique ID of each request.
Request Description	<td>\${request.description}</td>	Display the request description.	Displays the description of the request created by the staff.
Request Status	<td>\${request.status}</td>	Display the request status.	Displays the status of the request

			(e.g., pending, approved, rejected).
Request Created Date	<td>\${request.createdAt}</td>	Display the request's creation date.	Displays the date the request was created.